COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S1
DATE PERFORMED:	26/09/2026	**DATE SUBMITTED:**	Sep 29, 2026
NAME:	Formilleza, Garcia, Galvez, Igloso, Ignacio	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 [TREES ADT]

1. PROCEDURE OUTPUT

ILO A

HEADER FILE

```
#ifndef GT_H
#define GT_H

#include <iostream>

template <typename T>
class genTree{
    private:
        T key;
        genTree<T>* left;
        genTree<T>* right;
        genTree<T>* findData;
```

```
    public:
        // functions

        // CONSTRUCTOR
        genTree(T value);

        // DECONSTRUCTOR
        ~genTree();


        bool insertChild(T parentValue, T childValue);
        void display(int depth = 0);

};
```

```
template <typename T>
genTree<T>::genTree(T value)
    : key(value), left(nullptr), right(nullptr), findData(nullptr) {}

template <typename T>
genTree<T>::~genTree() {
    delete left;
    delete right;
}
```

```cpp
template <typename T>
void genTree<T>::find(T value) {
    if (key == value) {
        findData = this;
        return;
    }

    findData = nullptr;

    if (left != nullptr) {
        left->find(value);
        if (left->findData != nullptr) {
            findData = left->findData;
            return;
        }
    }
```

```cpp
    if (right != nullptr) {
        right->find(value);
        if (right->findData != nullptr) {
            findData = right->findData;
            return;
        }
    }
}
```

```cpp
template <typename T>
bool genTree<T>::insertChild(T parentValue, T childValue) {
    find(parentValue);
    genTree<T>* parent = findData;

    if (parent == nullptr) {
        std::cout << "Parent '" << parentValue << "' not found!" << std::endl;
        return false;
    }
```

```
    genTree<T>* newNode = new genTree<T>(childValue);

    if (parent->left == nullptr) {
        parent->left = newNode;
    } else {
        genTree<T>* sibling = parent->left;
        while (sibling->right != nullptr) {
            sibling = sibling->right;
        }
        sibling->right = newNode;
    }
    return true;
}
```

```
template <typename T>
void genTree<T>::display(int depth) {
    for (int i = 0; i < depth; i++) std::cout << "    ";
    std::cout << key << std::endl;

    if (left != nullptr) left->display(depth + 1);
    if (right != nullptr) right->display(depth);
}

#endif
```

MAIN DRIVER

```cpp
#include <iostream>
#include "GT.h"

int main() {
    // The root node itself IS the tree
    genTree<char> tree('A');

    // A's children: B, C, D, E, F, G
    tree.insertChild('A', 'B');
    tree.insertChild('A', 'C');
    tree.insertChild('A', 'D');
    tree.insertChild('A', 'E');
    tree.insertChild('A', 'F');
    tree.insertChild('A', 'G');
```

```cpp
    // D's child: H
    tree.insertChild('D', 'H');

    // E's children: I, J
    tree.insertChild('E', 'I');
    tree.insertChild('E', 'J');

    // F's children: K, L, M
    tree.insertChild('F', 'K');
    tree.insertChild('F', 'L');
    tree.insertChild('F', 'M');

    // G's child: N
    tree.insertChild('G', 'N');

    // J's children: P, Q
    tree.insertChild('J', 'P');
    tree.insertChild('J', 'Q');
```

```
    std::cout << "General Tree (indented by depth):" << std::endl;
    tree.display();

    return 0;
}
```

OUTPUT

```
General Tree (indented by depth):
A
   B
   C
   D
      H
   E
      I
      J
         P
         Q
   F
      K
      L
      M
   G
      N
```

Table 9.1

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1

G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	3
P	0	3
Q	0	3

Table 9.2

ILO B

HEADER FILE

```
#ifndef BST_H
#define BST_H

#include <iostream>

template <typename T>
class binarySearchTree{
    private:
        T key; // data

        // pointers for left and right
        binarySearchTree<T>* left;
        binarySearchTree<T>* right;

    public:
        // constructor
        binarySearchTree(T value);
```

```
    public:
        // constructor
        binarySearchTree(T value);

        //insert left and right
        void insert(T value);

        //traversing
        void preOrder();
        void inOrder();
        void postOrder();

        //searching
        bool findData(T value);
```

```cpp
template <typename T>
binarySearchTree<T>::binarySearchTree(T value){
    // Define key, and left and right pointers
    key = value;
    left = nullptr;
    right = nullptr;
}
```

```cpp
template <typename T>
void binarySearchTree<T>::insert(T value){
    // wheter to insert to the left
    if (value < key){
        if( left == nullptr){
            left = new binarySearchTree<T>(value);
        }
        else{
            left->insert(value);
        }
    }
    // or insert to the right
    else if (value > key){
        if (right == nullptr){
            right = new binarySearchTree<T>(value);
        }
        else{
            right->insert(value);
        }
    }
}
```

```cpp
template <typename T>
void binarySearchTree<T>::preOrder(){

    std::cout << key << " ";

    if (left != nullptr){
        left->preOrder();
    }

    if (right != nullptr){
        right->preOrder();
    }
}
```

```cpp
template <typename T>
void binarySearchTree<T>::inOrder(){

    if (left != nullptr){
        left->inOrder();
    }
    std::cout << key << " ";
    if (right != nullptr){
        right->inOrder();
    }

}
```

```cpp
template <typename T>
void binarySearchTree<T>::postOrder(){

    if (left != nullptr){
        left->postOrder();
    }
    if (right != nullptr){
        right->postOrder();
    }
    std::cout << key << " ";
}
```

```cpp
template <typename T>
bool binarySearchTree<T>::findData(T value) {
    if (value == key) {
        return true;
    }

    if (value < key && left != nullptr) {
        return left->findData(value);
    }

    if (value > key && right != nullptr) {
        return right->findData(value);
    }

    return false;
}

#endif
```

MAIN DRIVER

```cpp
#include <iostream>
#include "BST.h"


int main(){
    // NODE
    binarySearchTree<int> tree(20);

    tree.insert(30);
    tree.insert(70);
    tree.insert(50);
    tree.insert(40);
    tree.insert(60);
    tree.insert(80);
    tree.insert(10);
    tree.insert(15);

    std::cout << "PREORDER: \n";
    tree.preOrder();

    std::cout << "\n";

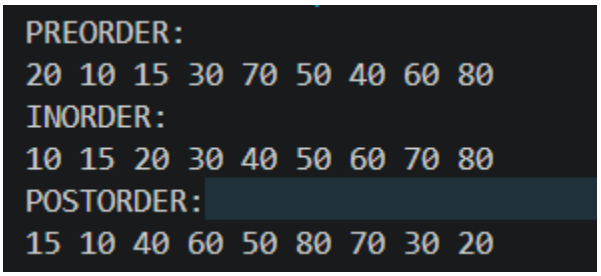
    std::cout << "INORDER: \n";
    tree.inOrder();
```

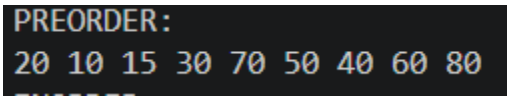
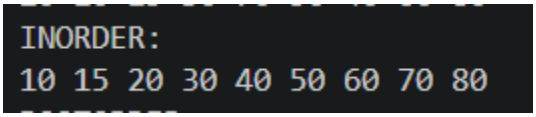
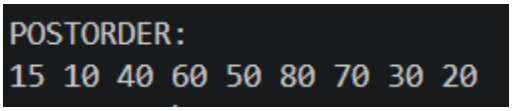
```cpp
std::cout << "POSTORDER: \n";
tree.postOrder();

std::cout << "\n";

// Searching test
std::cout << "=== Search Test ===" << std::endl;
int searchValue = 30;
if (tree.findData(searchValue)) {
    std::cout << "Value " << searchValue << " found in the tree!" << std::endl;
} else {
    std::cout << "Value " << searchValue << " not found in the tree." << std::endl;
}

return 0;
```

OUTPUT	

PREORDER	
INORDER	
POSTORDER	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SUPPLEMENTARY

HEADER FILE

```
#ifndef BINARYTREE_H
#define BINARYTREE_H

#include <iostream>

template <typename T>
class binarySearchTree {
    private:
        T key; // data

        // pointers for left and right
        binarySearchTree* left;
        binarySearchTree* right;

    public:
        // constructor
        binarySearchTree(T value);

        // destructor to clean up memory
        ~binarySearchTree();

        // insert left and right
        void insert(T value);

        // traversing
        void preOrder();
        void inOrder();
        void postOrder();

        // searching
        bool findData(T value);
};
```

```cpp
// --------------------
// Constructor function
// --------------------

template <typename T>
binarySearchTree<T>::binarySearchTree(T value) {
    // Define key, and left and right pointers
    key = value;
    left = nullptr;
    right = nullptr;
}

// -------------------
// Destructor function
// -------------------

template <typename T>
binarySearchTree<T>::~binarySearchTree() {
    delete left;
    delete right;
}
```

```cpp
template <typename T>
void binarySearchTree<T>::insert(T value) {
    // whether to insert to the left
    if (value < key) {
        if (left == nullptr) {
            left = new binarySearchTree(value);
        } else {
            left->insert(value);
        }
    }
    // or insert to the right
    else if (value > key) {
        if (right == nullptr) {
            right = new binarySearchTree(value);
        } else {
            right->insert(value);
        }
    }
}
```

```cpp
template <typename T>
void binarySearchTree<T>::preOrder() {
    std::cout << key << " ";

    if (left != nullptr) {
        left->preOrder();
    }

    if (right != nullptr) {
        right->preOrder();
    }
}
```

```cpp
// InOrder: Left-Root-Right
template <typename T>
void binarySearchTree<T>::inOrder() {
    if (left != nullptr) {
        left->inOrder();
    }

    std::cout << key << " ";

    if (right != nullptr) {
        right->inOrder();
    }
}
```

```cpp
// PostOrder: Left-Right-Root
template <typename T>
void binarySearchTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }

    if (right != nullptr) {
        right->postOrder();
    }

    std::cout << key << " ";
}
```

```
template <typename T>
bool binarySearchTree<T>::findData(T value) {
    if (value == key) {
        return true;
    }

    if (value < key && left != nullptr) {
        return left->findData(value);
    }

    if (value > key && right != nullptr) {
        return right->findData(value);
    }

    return false;
}

#endif
```

MAIN DRIVER

```
#include <iostream>
#include "binaryTree.h"

int main() {
    binarySearchTree root(2);

    int values[] = {3, 9, 18, 0, 1, 4, 5};

    for (int val : values) {
        root.insert(val);
    }

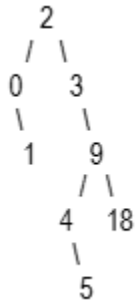
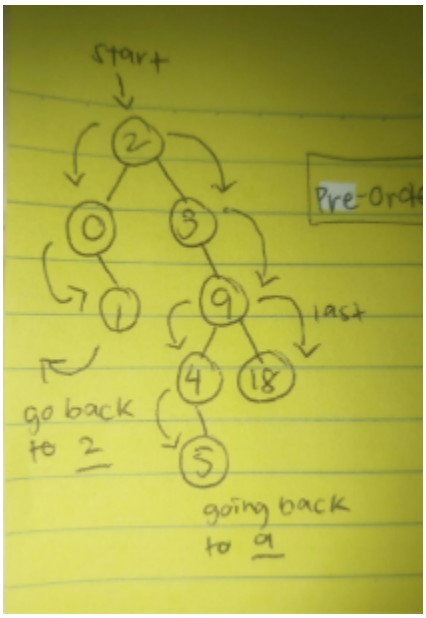
    std::cout << "Part 3: Binary Search Tree Traversals" << std::endl;

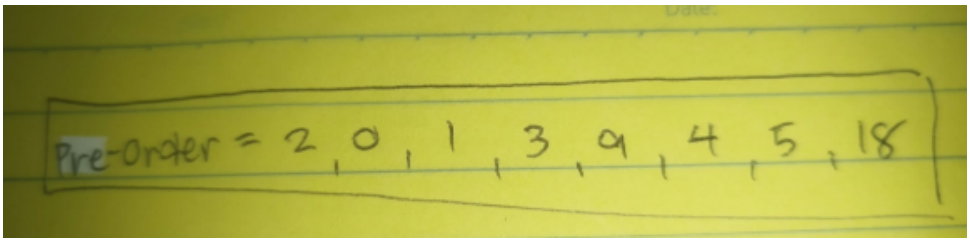
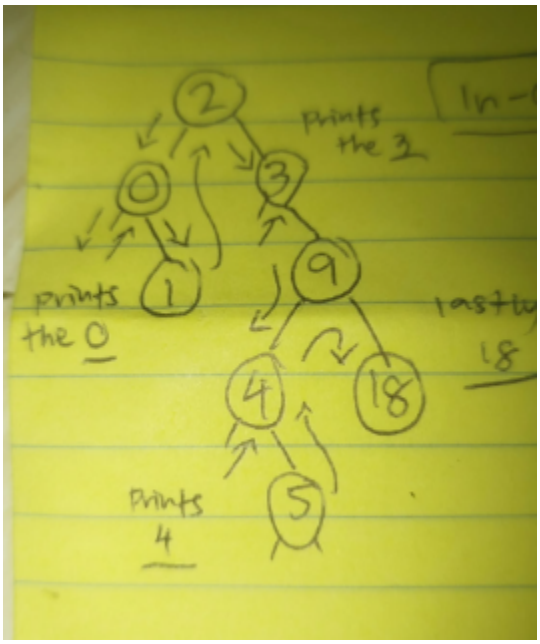
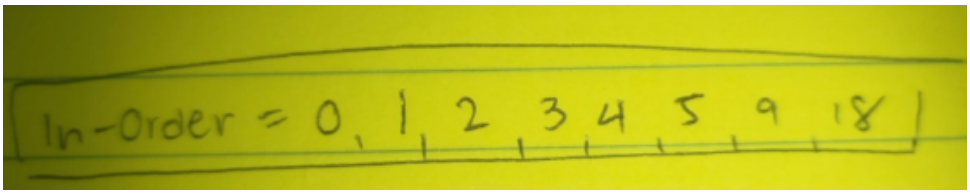
    std::cout << "In-order Traversal:   ";
    root.inOrder();
    std::cout << std::endl;

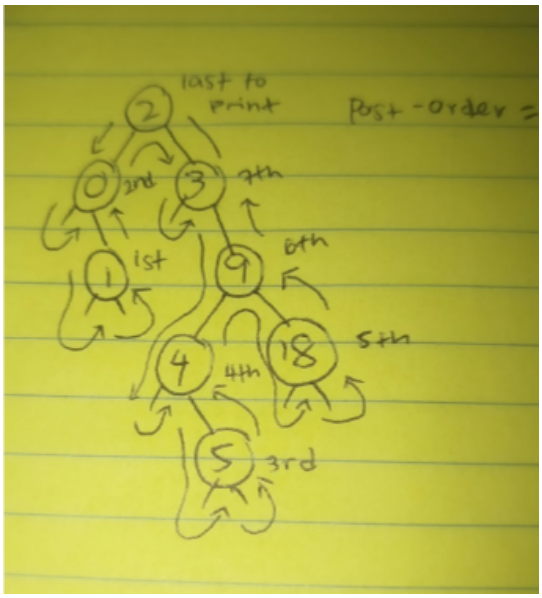
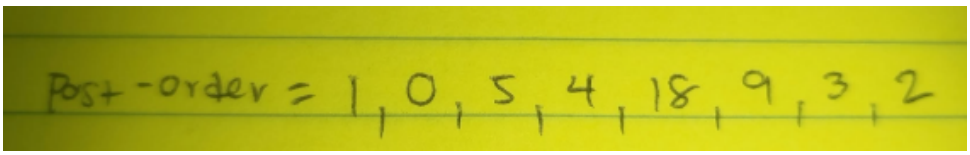
    std::cout << "Pre-order Traversal:  ";
    root.preOrder();
    std::cout << std::endl;
```

```
    std::cout << "Post-order Traversal: ";
    root.postOrder();
    std::cout << std::endl << std::endl;

    return 0;
}
```

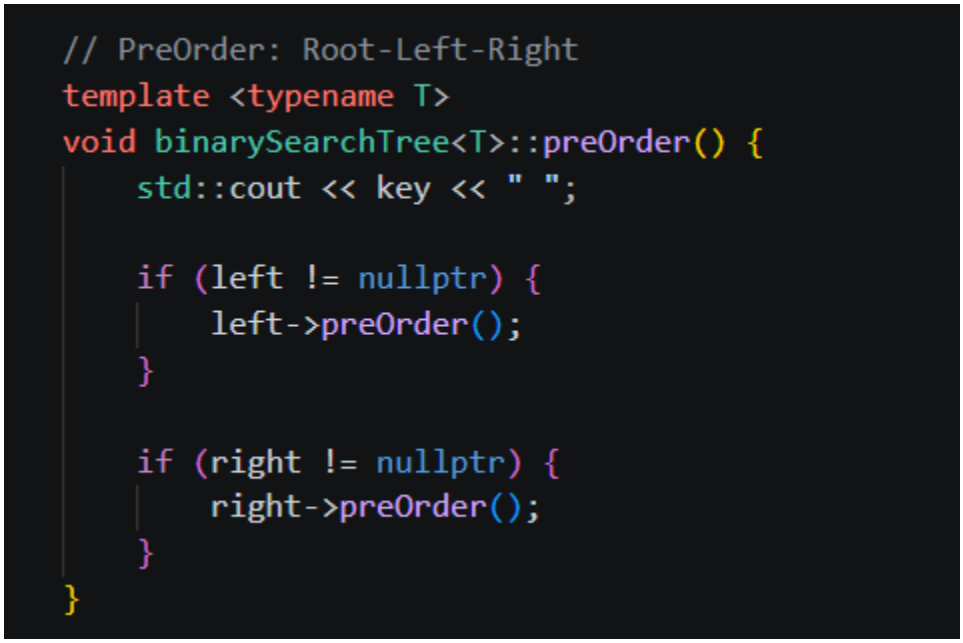
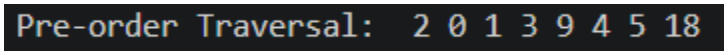
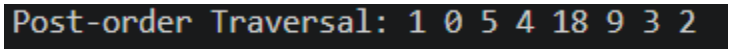
SCREENSHOT OF TREE DIAGRAM	
PREORDER SCREENSHOT	

	
INORDER SCREENSHOT	 

POSTORDER SCREENSHOT	 

INORDER TRAVERSAL CODE	```
	// InOrder: Left-Root-Right
	template <typename T>
	void binarySearchTree<T>::inOrder() {
	    if (left != nullptr) {
	        left->inOrder();
	    }
	
	    std::cout << key << " ";
	
	    if (right != nullptr) {
	        right->inOrder();
	    }
	}
	``

OUTPUT	``` In-order Traversal: 0 1 2 3 4 5 9 18 ```

PREORDER TRAVERSAL CODE	
OUTPUT	
POSTORDER CODE	
OUTPUT	

```
// PreOrder: Root-Left-Right
template <typename T>
void binarySearchTree<T>::preOrder() {
    std::cout << key << " ";

    if (left != nullptr) {
        left->preOrder();
    }

    if (right != nullptr) {
        right->preOrder();
    }
}
```

```
Pre-order Traversal:  2 0 1 3 9 4 5 18
```

```
// PostOrder: Left-Right-Root
template <typename T>
void binarySearchTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }

    if (right != nullptr) {
        right->postOrder();
    }

    std::cout << key << " ";
}
```

```
Post-order Traversal: 1 0 5 4 18 9 3 2
```

CONCLUSION

Provide the following:
Summary of lessons learned
Analysis of the procedure
Analysis of the supplementary activity
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.